

JUNIOR KUBERNETES ADMINISTRATOR PROJECTS

From Kubernetes Fundamentals to
Practical Cluster Administration

kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
node-1	Ready	control-plane	10d	v1.29.3
node-2	Ready	<none>	10d	v1.29.3
node-3	Ready	<none>	10d	v1.29.3

NAMESPACES

default	Active
kube-system	Active
dev	Active
staging	Active
production	Active

APPLICATION ARCHITECTURE

Frontend

Backend API

Database

RESOURCE OVERVIEW

NODES	3
PODS	24
DEPLOYMENTS	8
SERVICES	12



PRACTICAL
PROJECTS



REAL-WORLD
SCENARIOS



TROUBLESHOOTING
FOCUSED



SECURE &
RELIABLE



CLUSTER
OPERATIONS



ADMIN SKILLS
CHECKLIST

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



JUNIOR KUBERNETES ADMINISTRATOR PROJECT LAB



FROM KUBERNETES FUNDAMENTALS TO PRACTICAL CLUSTER ADMINISTRATION



WHAT YOU WILL BUILD

- A WORKING KUBERNETES CLUSTER
- REAL-WORLD APPLICATIONS
- SECURE AND CONFIGURED WORKLOADS
- PERSISTENT STORAGE
- ROLLING UPDATES AND ROLLBACKS
- MONITORING AND HEALTH CHECKS
- TROUBLESHOOTING SKILLS
- A PRODUCTION-STYLE ENVIRONMENT



REQUIRED TOOLS



MINIKUBE OR KIND

Run a local Kubernetes cluster for practice.



KUBECTL

Command-line tool to manage Kubernetes clusters.



DOCKER

Build and run container images.



YAML EDITOR

Create and edit Kubernetes manifests.



GIT

Version control for your manifests and projects.



HOW THE 20-PAGE PROJECT JOURNEY WORKS

- 1 **BUILD** – Deploy and configure core Kubernetes resources.
- 2 **OPERATE** – Manage workloads, updates, storage, and security.
- 3 **OBSERVE** – Monitor, inspect, and validate cluster behavior.
- 4 **TROUBLESHOOT** – Investigate and fix real-world issues.
- 5 **SECURE** – Apply RBAC and security best practices.
- 6 **MASTER** – Recover a broken environment in the final project.



EACH PROJECT BUILDS YOUR CONFIDENCE AND SKILLS
TOWARD BECOMING A RELIABLE KUBERNETES ADMINISTRATOR.



FINAL ADMINISTRATOR SKILLS CHECKLIST

- CLUSTER SETUP AND MANAGEMENT ☒
- WORKLOAD DEPLOYMENT AND MANAGEMENT ☒
- SERVICES AND NETWORKING ☒
- CONFIGMAPS AND SECRETS ☒
- STORAGE AND PERSISTENCE ☒
- ROLLING UPDATES AND ROLLBACKS ☒
- HEALTH CHECKS AND PROBES ☒
- RESOURCE MANAGEMENT ☒
- JOBS AND CRONJOBS ☒
- SCHEDULING AND PLACEMENT ☒
- TROUBLESHOOTING AND LOG ANALYSIS ☒
- RBAC AND ACCESS CONTROL ☒
- CLUSTER MONITORING AND HEALTH CHECKS ☒
- INCIDENT RESPONSE AND RECOVERY ☒



BY THE END OF THIS 20-PAGE LAB, YOU WILL HAVE PRACTICAL, HANDS-ON EXPERIENCE
TO WORK CONFIDENTLY AS A JUNIOR KUBERNETES ADMINISTRATOR.



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

VERIQTA SOCIAL

PROJECT 1

BUILD YOUR FIRST
KUBERNETES CLUSTER

1



CREATE A LOCAL KUBERNETES CLUSTER

Use Minikube or Kind to create a local Kubernetes cluster on your machine.

2



VERIFY THE CONTROL PLANE

Ensure the control plane components are running and healthy.

3



INSPECT CLUSTER INFORMATION

View cluster details, endpoints, and configuration.

4



LIST NODES

Check the nodes that are part of your cluster.

5



CHECK KUBERNETES VERSION

Verify both client (kubectl) and server versions.

6



EXPLORE SYSTEM NAMESPACES

Look at the default system namespaces created by Kubernetes.

7



UNDERSTAND WHAT IS RUNNING

Explore the system Pods, Deployments, and core components.

PRACTICE COMMANDS

>_ kubectl cluster-info

Display information about the cluster endpoints and services.

```
$ kubectl cluster-info
Kubernetes control plane is
running at https://127.0.0.1:6443
CoreDNS is running at https://
//127.0.0.1:6443/api/v1/namespaces/
kube-system/services/kube-dns:dns/proxy
```

>_ kubectl get nodes

List all nodes in the cluster and their status.

```
$ kubectl get nodes
NAME      STATUS  ROLES    AGE  VERSION
minikube  Ready   control-plane  2m   v1.29.3
```

>_ kubectl get namespaces

List all namespaces in the cluster.

```
$ kubectl get namespaces
NAME      STATUS  AGE
default   Active  2m
kube-system  Active  2m
kube-public  Active  2m
kube-node-lease  Active  2m
```

>_ kubectl get pods -A

List all Pods in all namespaces.

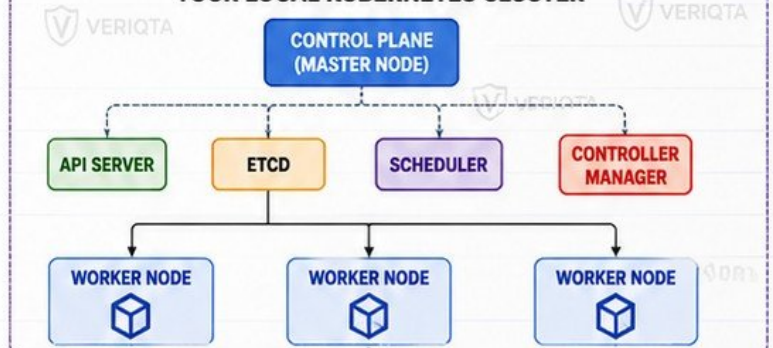
```
$ kubectl get pods -A
NAMESPACE  NAME                                READY  STATUS  RESTARTS  AGE
kube-system  coredns-0000000000                 1/1    Running  0          2m
kube-system  etcd-minikube                       1/1    Running  0          2m
kube-system  kube-apiserver-minikube             1/1    Running  0          2m
kube-system  kube-controller-manager             1/1    Running  0          2m
kube-system  kube-proxy-0000000000              1/1    Running  0          2m
kube-system  kube-scheduler-minikube            1/1    Running  0          2m
```

KEY THINGS TO OBSERVE

- ✓ THE CONTROL PLANE IS RUNNING AND HEALTHY
- ✓ NODES ARE IN READY STATE
- ✓ CORE SYSTEM NAMESPACES EXIST
- ✓ SYSTEM PODS ARE RUNNING
- ✓ CLUSTER ENDPOINTS ARE ACCESSIBLE
- ✓ YOU CAN EXECUTE kubectl COMMANDS SUCCESSFULLY

WHAT YOU ARE BUILDING

YOUR LOCAL KUBERNETES CLUSTER



TIP This is the foundation of everything in Kubernetes. Before deploying applications, make sure your cluster is healthy and all system components are running.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 2

DEPLOY YOUR FIRST
APPLICATION

DEPLOY AN NGINX APPLICATION USING A DEPLOYMENT

1 CREATE AN NGINX DEPLOYMENT



Create a Deployment to run the NGINX web server.

2 DEFINE REPLICAS



Set the desired number of replicas to ensure high availability.

3 APPLY THE MANIFEST



Apply the Deployment YAML manifest to the cluster.

4 INSPECT PODS



Check that Pods are created and running as expected.

5 INSPECT THE DEPLOYMENT



Review the Deployment status, replicas, strategy, and events.

6 VERIFY APPLICATION STATUS



Access the application and confirm it is running successfully.

7 DELETE AND RECREATE THE WORKLOAD



Delete the Deployment and recreate it to understand the full lifecycle.

8 UNDERSTAND THE RELATIONSHIP



Understand how Deployment manages ReplicaSets, which manage Pods.

DEPLOYMENT → REPLICASET → PODS



SAMPLE DEPLOYMENT MANIFEST

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

nginx-deployment.yaml

PRACTICE COMMANDS

> kubectl apply -f nginx-deployment.yaml	Apply the Deployment manifest.
> kubectl get pods	List Pods created by the Deployment.
> kubectl get deployment	View Deployment details and status.
> kubectl describe deployment nginx-deployment	Inspect Deployment events and conditions.
> kubectl delete deployment nginx-deployment	Delete the Deployment.
> kubectl apply -f nginx-deployment.yaml	Recreate the Deployment.

WHAT YOU WILL LEARN

- ✓ HOW TO DEPLOY AN APPLICATION USING A DEPLOYMENT
- ✓ HOW REPLICAS ENSURE HIGH AVAILABILITY
- ✓ HOW DEPLOYMENTS MANAGE REPLICASETS AND PODS
- ✓ HOW TO MANAGE THE LIFECYCLE OF APPLICATIONS
- ✓ HOW TO VERIFY AND MAINTAIN APPLICATION HEALTH

SUCCESS CHECKLIST

- ✓ NGINX DEPLOYMENT CREATED SUCCESSFULLY
- ✓ PODS ARE RUNNING AND HEALTHY
- ✓ DEPLOYMENT STATUS IS AVAILABLE
- ✓ APPLICATION IS ACCESSIBLE
- ✓ WORKLOAD DELETED AND RECREATED SUCCESSFULLY
- ✓ UNDERSTAND DEPLOYMENT TO REPLICASET TO POD RELATIONSHIP

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 3

EXPOSE AN APPLICATION
WITH A SERVICE

EXPOSE YOUR NGINX DEPLOYMENT USING A SERVICE (CLUSTERIP)

1



CREATE A SERVICE

Create a Kubernetes Service to expose your NGINX Deployment.

2



WORK WITH CLUSTERIP

Use the default ClusterIP type to expose the application internally.

3



EXPOSE THE NGINX APPLICATION

Map the Service to the Deployment using the correct selector and port.

4



INSPECT SERVICE ENDPOINTS

Verify that the Service has endpoints pointing to healthy Pods.

5



TEST CONNECTIVITY

Access the application from inside the cluster using the Service.

6



UNDERSTAND POD IP VS SERVICE IP

Compare Pod IP addresses with the Service ClusterIP.

7



VERIFY TRAFFIC REACHES HEALTHY PODS

Ensure traffic is load-balanced only to healthy Pods behind the Service.

SAMPLE SERVICE MANIFEST (CLUSTERIP)

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
  labels:
    app: nginx
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - name: http
      port: 80
      targetPort: 80
```

nginx-service.yaml

PRACTICE COMMANDS

>_	kubectl apply -f nginx-service.yaml	Create the Service.
>_	kubectl get svc	List Services in the namespace.
>_	kubectl get endpoints nginx-service	Check endpoints behind the Service.
>_	kubectl get pods -o wide	View Pod IP addresses.
>_	kubectl exec -it <pod-name> -- bash	Access a Pod shell.
>_	curl http://nginx-service	Test connectivity to the Service from inside the cluster.

POD IP VS SERVICE IP

POD IP (DIRECT)

Pod A 10.244.1.5
Pod B 10.244.1.6
Pod C 10.244.1.7

PODS ARE DYNAMIC
AND CAN CHANGE

VS

SERVICE IP (STABLE)

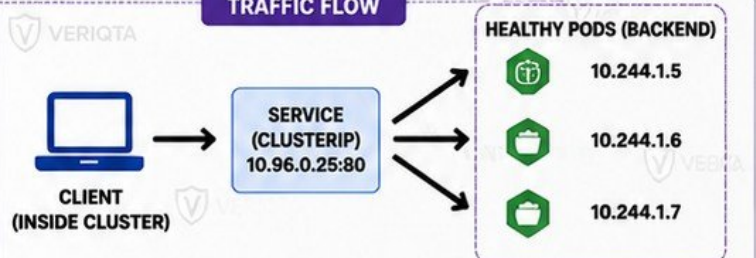
Service Name: nginx-service
ClusterIP: 10.96.0.25
Port: 80

SERVICE IP IS STABLE
AND DOES NOT CHANGE

KEY TAKEAWAYS

- ✓ SERVICES PROVIDE A STABLE ENDPOINT FOR APPLICATIONS
- ✓ CLUSTERIP KEEPS TRAFFIC INTERNAL TO THE CLUSTER
- ✓ SERVICES LOAD-BALANCE TRAFFIC TO HEALTHY PODS
- ✓ ENDPOINTS SHOW WHICH PODS RECEIVE TRAFFIC
- ✓ POD IPs CHANGE, SERVICE IPs REMAIN STABLE

TRAFFIC FLOW



SUCCESS CHECKLIST

- ✓ SERVICE CREATED SUCCESSFULLY
- ✓ SERVICE TYPE IS CLUSTERIP
- ✓ SERVICE HAS HEALTHY ENDPOINTS
- ✓ CONNECTIVITY TEST TO SERVICE IS SUCCESSFUL
- ✓ TRAFFIC REACHES HEALTHY PODS ONLY

VERIQTA SOCIAL/VERIQTA/VERIQTA/VERIQTA/VERIQTA/VERIQTA/VERIQTA

PROJECT 4

MANAGE APPLICATIONS
WITH NAMESPACES

ORGANIZE AND ISOLATE APPLICATIONS IN A SHARED CLUSTER

1



CREATE NAMESPACES

Create development, staging, and production namespaces.

```
kubectl create namespace dev
kubectl create namespace staging
kubectl create namespace prod
```

2

DEPLOY WORKLOADS
INTO NAMESPACES

Deploy applications into specific namespaces.

```
kubectl apply -f nginx-deploy.yaml -n dev
kubectl apply -f nginx-deploy.yaml -n staging
kubectl apply -f nginx-deploy.yaml -n prod
```

3

LIST NAMESPACE
RESOURCES

View all resources inside a namespace.

```
kubectl get all -n dev
kubectl get all -n staging
kubectl get all -n prod
```

4

SET A DEFAULT
NAMESPACE

Set the default namespace for your kubectl context.

```
kubectl config set-context --current --namespace=dev
```

5

DELETE NAMESPACE
RESOURCES SAFELY

Delete all resources in a namespace by deleting the namespace.

```
kubectl delete namespace staging
```

CAUTION: This will delete ALL resources inside the namespace.

6

PRACTICE ORGANIZING
A SHARED CLUSTER

Use namespaces to isolate teams, environments, and applications.

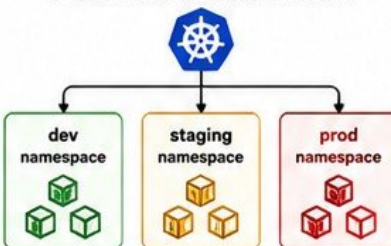


NAMESPACE OVERVIEW

NAMESPACE	PURPOSE	TYPICAL USE
dev	Development	Feature development and unit testing
staging	Testing	Integration testing and pre-production validation
prod	Production	Live production applications
default	Default	Used when no namespace is specified

EXAMPLE ARCHITECTURE

SHARED KUBERNETES CLUSTER



BENEFITS OF NAMESPACES

- ✓ ISOLATE ENVIRONMENTS
- ✓ PREVENT RESOURCE CONFLICTS
- ✓ MANAGE ACCESS WITH RBAC
- ✓ ORGANIZE APPLICATIONS
- ✓ SIMPLIFY RESOURCE QUOTAS
- ✓ IMPROVE CLUSTER GOVERNANCE
- ✓ SUPPORT MULTI-TEAM WORKFLOWS

PRACTICE COMMANDS

> kubectl get namespaces	List all namespaces.	> kubectl config set-context --current --namespace=dev	Set default namespace.
> kubectl describe ns dev	View namespace details.	> kubectl get ns	List namespaces.
> kubectl get pods -n dev	List Pods in the dev namespace.	> kubectl delete ns staging	Delete a namespace.
> kubectl get all -A	List all resources in all namespaces.	> kubectl api-resources --namespaced=true	List namespaced resources.



TIP

Always use namespaces to organize applications in shared clusters. It improves security, isolation, and cluster management.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 5

CONFIGURE AN APPLICATION
WITH CONFIGMAPS

MANAGE APPLICATION CONFIGURATION THE KUBERNETES WAY

1



CREATE A CONFIGMAP

Create a ConfigMap to store application configuration.

```
kubectl create configmap  
app-config --from-literal=APP_NAME=NGINX  
--from-literal=APP_PORT=8080  
--from-literal=ENV=DEV
```

2

STORE APPLICATION
CONFIGURATIONStore key-value pairs or files
inside a ConfigMap.

```
apiVersion: v1  
kind: ConfigMap  
metadata:  
  name: app-config  
data:  
  APP_NAME: "NGINX"  
  APP_PORT: "8080"  
  ENV: "DEV"
```

3

INJECT CONFIG AS
ENVIRONMENT VARIABLESUse ConfigMap values as
environment variables in a Pod.

```
env:  
- name: APP_NAME  
  valueFrom:  
    configMapKeyRef:  
      name: app-config  
      key: APP_NAME  
- name: APP_PORT  
  valueFrom:  
    configMapKeyRef:  
      name: app-config  
      key: APP_PORT
```

4

MOUNT CONFIGMAP
AS FILESMount ConfigMap data
as files in a volume.

```
volumes:  
- name: config-volume  
  configMap:  
    name: app-config  
  
volumeMounts:  
- name: config-volume  
  mountPath: /etc/app/config
```

5



UPDATE CONFIGURATION

Update the ConfigMap
when configuration changes.

```
kubectl create configmap  
app-config --from-literal=APP_NAME=NGINX  
--from-literal=APP_PORT=9090  
--from-literal=ENV=PROD --dry-run=client  
-o yaml | kubectl apply -f -
```

PODS WILL AUTOMATICALLY PICK UP
CHANGES FOR ENV VARS. FOR FILE
MOUNTS, RESTART THE POD.

6



VERIFY INSIDE THE CONTAINER

Check that configuration values
are available inside the Pod.

```
# Check environment variables  
kubectl exec -it <pod-name> -- env |  
grep APP_  
  
# Check mounted files  
kubectl exec -it <pod-name> --  
cat /etc/app/config/APP_NAME
```

7

VERIFY APPLICATION
BEHAVIORConfirm the application is using
the correct configuration.

```
kubectl logs <pod-name>  
  
curl http://<pod-ip>:8080  
  
# Or inside the container  
curl http://localhost:$APP_PORT
```

8

TROUBLESHOOT INCORRECT
CONFIGMAP REFERENCESCommon issues and how
to fix them.

- CONFIGMAP NOT FOUND
 - Check name spelling
 - Ensure it exists in the namespace
- KEY NOT FOUND
 - Verify key name in the ConfigMap
- POD NOT RELOADING CHANGES
 - Restart the Pod for file mounts
 - Env vars update on restart only
- WRONG NAMESPACE
 - Check the namespace context

CONFIGMAP WORKFLOW

CREATE
CONFIGMAPSTORE
CONFIGURATIONUSE IN POD
(ENV OR FILES)UPDATE
CONFIGURATIONVERIFY &
TROUBLESHOOT

CONFIGMAP USAGE OPTIONS

AS ENVIRONMENT
VARIABLES

```
POD  
APP_NAME  
APP_PORT  
ENV
```

AS FILES
(VOLUME MOUNT)

```
POD  
/etc/app/config  
APP_NAME  
APP_PORT  
ENV
```

BEST PRACTICES

- ✓ USE CONFIGMAPS FOR NON-SENSITIVE DATA
- ✓ KEEP KEYS SIMPLE AND CONSISTENT
- ✓ USE NAMESPACES FOR ISOLATION
- ✓ VERSION YOUR CONFIGURATION
- ✓ RESTART PODS FOR FILE MOUNT CHANGES
- ✓ MONITOR APPLICATION AFTER UPDATES

PRACTICE COMMANDS

>> kubectl get configmaps	List all ConfigMaps.	>> kubectl exec -it <pod-name> -- cat /etc/app/config/APP_NAME	Check mounted file content.
>> kubectl describe configmap app-config	View ConfigMap details.	>> kubectl apply -f configmap.yaml	Apply a ConfigMap manifest.
>> kubectl get pods	List Pods using the ConfigMap.	>> kubectl delete configmap app-config	Delete a ConfigMap.
>> kubectl exec -it <pod-name> -- env grep APP_	Check env vars inside the Pod.	>> kubectl rollout restart deployment <deployment-name>	Restart Deployment to reload config.



TIP

CONFIGMAPS HELP YOU MANAGE APPLICATION CONFIGURATION IN A CLEAN, REUSABLE, AND
ENVIRONMENT-FRIENDLY WAY.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 6



VERIQTA

MANAGE APPLICATION
SECRETS

VERIQTA



SECURELY STORE AND USE SENSITIVE INFORMATION IN KUBERNETES

VERIQTA

1

CREATE KUBERNETES
SECRETS

Create a Secret to store sensitive information securely.

```
kubectl create secret generic db-secret \
--from-literal=DB_USER=admin \
--from-literal=DB_PASSWORD=StrongPass123 \
--from-literal=DB_HOST=db-service \
--namespace=dev
```

VERIQTA

VERIQTA

2

REFERENCE SECRETS
FROM PODS

Use Secrets in Pod manifests as environment variables or volumes.

```
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_PASSWORD
```

3

INJECT CREDENTIALS AS
ENVIRONMENT VARIABLES

Expose Secret values as environment variables.

```
env:
- name: DB_USER
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_USER
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_PASSWORD
```

4

MOUNT SECRETS
AS FILES

Mount Secret values as files inside the container.

```
volumes:
- name: secret-volume
  secret:
    secretName: db-secret

volumeMounts:
- name: secret-volume
  mountPath: /etc/secrets
  readOnly: true
```

5



INSPECT SECRET REFERENCES

Inspect Secrets and verify how they are used by Pods.

```
kubectl get secrets -n dev
kubectl describe secret db-secret -n dev
kubectl get pods -n dev -o yaml | \
grep -A5 -i secret
```

6

AVOID HARDCODING
CREDENTIALS

Never store passwords or tokens directly in manifests.

AVOID THIS

```
password: "StrongPass123"
token: "abcd1234efgh"
```

USE SECRETS

```
valueFrom:
  secretKeyRef:
    name: db-secret
    key: DB_PASSWORD
```

7

TROUBLESHOOT MISSING
SECRET ERRORS

Identify and fix common Secret-related issues.

- **SECRET NOT FOUND**
 - Check Secret name spelling
 - Check namespace
- **WRONG KEY**
 - Verify the key exists in the Secret
- **PERMISSION ISSUES (RBAC)**
 - Ensure the ServiceAccount has access
- **POD NOT RELOADING UPDATED SECRET**
 - Restart the Pod to reload the Secret

SECRET TYPES

TYPE	DESCRIPTION
Opaque	Default type for arbitrary key-value secrets.
kubernetes.io/dockerconfigjson	Stores Docker registry credentials.
kubernetes.io/tls	Stores TLS certificate and key.
kubernetes.io/basic-auth	Stores basic authentication credentials.

SECRET WORKFLOW

CREATE
SECRETREFERENCE IN
PODUSE AS ENV
OR FILEVERIFY &
TROUBLESHOOT

KEY BEST PRACTICES

- ✓ USE SECRETS FOR ALL SENSITIVE DATA
- ✓ NEVER COMMIT SECRETS TO GIT
- ✓ USE NAMESPACES TO ISOLATE SECRETS
- ✓ LIMIT RBAC ACCESS TO SECRETS
- ✓ ROTATE SECRETS REGULARLY
- ✓ RESTART PODS AFTER SECRET UPDATES

PRACTICE COMMANDS

> kubectl create secret generic my-secret --from-literal=password123	Create a generic Secret.	> kubectl exec -it <pod-name> -- env grep DB_	Check env vars inside a Pod.
> kubectl get secrets	List all Secrets.	> kubectl exec -it <pod-name> -- ls -l /etc/secrets	Check mounted Secret files.
> kubectl describe secret my-secret	View Secret details.	> kubectl apply -f deployment.yaml	Apply Pod manifest using Secrets.
> kubectl get pods -o yaml grep -A5 secret	Find Secret references in a Pod manifest.	> kubectl delete secret my-secret	Delete a Secret.



TIP

SECRETS ENCRYPT DATA AT REST IN ETCD, BUT ACCESS CONTROL IS YOUR RESPONSIBILITY. FOLLOW THE PRINCIPLE OF LEAST PRIVILEGE.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



PAGE 8 - PROJECT 7

PERFORM A ROLLING APPLICATION UPDATE



OBJECTIVE

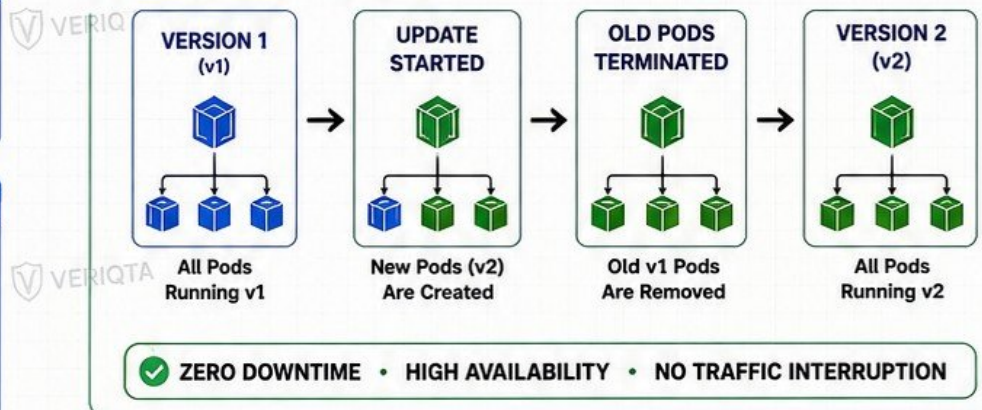
Update an application to a new version using a rolling update, monitor the rollout, inspect the history, and roll back if needed.



PROJECT STEPS

- 1 DEPLOY APPLICATION VERSION 1
- 2 CHANGE THE CONTAINER IMAGE
- 3 PERFORM A ROLLING UPDATE
- 4 WATCH ROLLOUT PROGRESS
- 5 INSPECT ROLLOUT HISTORY
- 6 ROLL BACK A FAILED DEPLOYMENT

ROLLING UPDATE FLOW



VERSION 1 (DEPLOYMENT YAML)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginx:1.21 # Version 1
          ports:
            - containerPort: 80
```

UPDATE IMAGE
TO NEW VERSION

VERSION 2 (UPDATED DEPLOYMENT YAML)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: web
          image: nginx:1.25 # Version 2 (New)
          ports:
            - containerPort: 80
```

PRACTICE - KUBECTL COMMANDS



1. CHECK ROLLOUT STATUS

```
kubectl rollout status deployment/web-app
-n default
```

MONITOR THE PROGRESS OF THE ROLLING UPDATE.



2. VIEW ROLLOUT HISTORY

```
kubectl rollout history deployment/web-app
-n default
```

VIEW REVISION HISTORY OF THE DEPLOYMENT.



3. ROLL BACK TO PREVIOUS REVISION

```
kubectl rollout undo deployment/web-app
-n default
```

ROLL BACK TO THE PREVIOUS WORKING REVISION.



KEY POINTS

- Rolling updates ensure zero downtime.
- Kubernetes replaces old Pods with new Pods gradually.
- If something goes wrong, roll back immediately.
- Always test your application after updates.



IMPORTANT NOTES

- Ensure readiness probes are configured correctly for smooth updates.
- Monitor resource usage during updates.
- Always keep deployment revisions clean and meaningful.



REAL-WORLD TIP

Always update in a staging environment first. Once stable, promote the image to production.



VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



PAGE 9 - PROJECT 8

CONFIGURE HEALTH CHECKS



OBJECTIVE

Ensure your applications remain healthy and available by configuring and troubleshooting Kubernetes health checks.



PROJECT STEPS

- 1 ADD LIVENESS PROBES
- 2 ADD READINESS PROBES
- 3 UNDERSTAND STARTUP PROBES
- 4 SIMULATE AN UNHEALTHY APPLICATION
- 5 OBSERVE KUBERNETES RESPONSE
- 6 TROUBLESHOOT FAILED PROBES
- 7 FIX INCORRECT PATHS, PORTS, AND TIMING VALUES

THE THREE PROBES

LIVENESS PROBE



Detects if the container is running.
If it fails, Kubernetes restarts the container.

READINESS PROBE



Detects if the container is ready to serve traffic.
If it fails, the Pod is removed from Service endpoints.

STARTUP PROBE



Used for slow-starting applications.
It disables liveness and readiness checks until startup succeeds.

EXAMPLE: PROBES IN A DEPLOYMENT

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
  successThreshold: 1
---
readinessProbe:
  httpGet:
    path: /ready
    port: 8080
  initialDelaySeconds: 5
  periodSeconds: 5
  timeoutSeconds: 2
  failureThreshold: 3
  successThreshold: 1
---
startupProbe:
  httpGet:
    path: /startup
    port: 8080
  failureThreshold: 30
  periodSeconds: 10
  timeoutSeconds: 2
```

PROBE BEHAVIOR

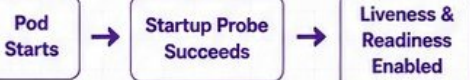
LIVENESS PROBE FAILURE



READINESS PROBE FAILURE



STARTUP PROBE IN USE



SIMULATE FAILURES

- Wrong path: /wrong-path
- Wrong port: 9999
- Slow response
- Application crash
- Database dependency down



OBSERVE KUBERNETES RESPONSE

- Check Pod status: Running, NotReady, CrashLoopBackOff
- Describe Pod events
- Check restart count
- Check probe logs from the application



TROUBLESHOOT FAILED PROBES

PROBLEM SYMPTOM	POSSIBLE CAUSE	TROUBLESHOOT COMMANDS	FIX
Liveness probe fails repeatedly	Wrong path, port, or application error	kubectl describe pod <pod-name> kubectl logs <pod-name>	Correct path/port, fix application error
Readiness probe fails continuously	App not ready, slow startup, wrong endpoint	kubectl describe pod <pod-name> kubectl logs <pod-name>	Fix readiness logic or increase delays
Startup probe fails	Application takes too long to start	kubectl describe pod <pod-name>	Increase failureThreshold or adjust periodSeconds
Pod restarting (CrashLoopBackOff)	Liveness probe killing the container	kubectl get pods -w kubectl describe pod <pod-name>	Fix app or increase initialDelaySeconds



KEY TAKEAWAYS

- Liveness keeps the application running.
- Readiness protects users from unready Pods.
- Startup prevents premature failures.



PRACTICE COMMANDS

```
kubectl get pods -w
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl rollout restart deployment/<name>
```

PROBE TIMING GUIDE



- **initialDelaySeconds**: Wait time before first probe.
- **periodSeconds**: How often the probe runs.
- **timeoutSeconds**: How long to wait for response.
- **failureThreshold**: Consecutive failures before action.
- **successThreshold**: Consecutive successes before marking ready.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



PAGE 10 - PROJECT 9

CONTROL CPU AND MEMORY



OBJECTIVE

Learn how to manage CPU and memory resources to ensure application stability, prevent resource exhaustion, and maintain cluster health.



PROJECT STEPS

- 1 CONFIGURE RESOURCE REQUESTS
- 2 CONFIGURE RESOURCE LIMITS
- 3 INSPECT POD RESOURCE CONFIGURATION
- 4 TRIGGER A MEMORY-RELATED FAILURE
- 5 INVESTIGATE OOMKILLED
- 6 IDENTIFY RESOURCE PRESSURE
- 7 UNDERSTAND PENDING PODS CAUSED BY INSUFFICIENT RESOURCES

1 CONFIGURE RESOURCE REQUESTS AND LIMITS

EXAMPLE: POD WITH CPU AND MEMORY REQUESTS & LIMITS

```
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
    - name: demo
      image: k8s.gcr.io/stress-ng:latest
      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
      command: ["stress-ng", "--vm", "1", "--vm-bytes",
        "400M", "--vm-hang", "0"]
```



KEY CONCEPTS

- ✓ **REQUESTS**
Minimum resources the Pod needs to run.
- ✓ **LIMITS**
Maximum resources the Pod can use.
- ✓ **REQUESTS** affect scheduling.
- ✓ **LIMITS** enforce usage constraints.
- ✓ 合理配置资源 = 稳定的应用 + 健康的集群。

2 INSPECT POD RESOURCE CONFIGURATION

VIEW POD RESOURCES

```
kubectl describe pod resource-demo
```

VIEW NODE RESOURCES

```
kubectl top nodes
```

VIEW POD RESOURCE USAGE

```
kubectl top pods
```

Dofortimus



3 RESOURCE UNITS REFERENCE

RESOURCE	MEANING	EXAMPLES
CPU	Measured in cores or millicores	500m = 0.5 CPU 1000m = 1 CPU 2 = 2 CPU
MEMORY	Measured in bytes	128Mi, 256Mi, 512Mi 1Gi, 2Gi



TRIGGER A MEMORY-RELATED FAILURE

Deploy a Pod that tries to use more memory than its limit.

```
stress-ng --vm 1 --vm-bytes 800M --vm-hang 0
```



When the container exceeds its memory limit, Kubernetes will terminate it.



INVESTIGATE OOMKILLED

CHECK POD STATUS

```
kubectl get pod resource-demo
```

DESCRIBE THE POD

```
kubectl describe pod resource-demo
```

CHECK CONTAINER STATE

Look for:
State: Terminated
Reason: OOMKilled

OOM

OOMKilled = Out Of Memory



IDENTIFY RESOURCE PRESSURE

CHECK NODE CONDITIONS

```
kubectl describe node <node-name>
```

POSSIBLE CONDITIONS

- MemoryPressure True
- DiskPressure False
- PIDPressure False



Resource pressure can prevent Pods from starting and cause existing Pods to be evicted.

7 UNDERSTAND PENDING PODS CAUSED BY INSUFFICIENT RESOURCES

SCENARIO	REASON	WHAT YOU SEE	HOW TO CHECK	HOW TO FIX
CPU Insufficient	Not enough CPU available	Pod status: Pending Events: Insufficient cpu	kubectl describe pod <pod-name>	Free up CPU or reduce requests/limits
Memory Insufficient	Not enough memory available	Pod status: Pending Events: Insufficient memory	kubectl describe pod <pod-name>	Free up memory or reduce requests/limits
Both Insufficient	Not enough CPU and memory	Pod status: Pending Events: Insufficient cpu, Insufficient memory	kubectl describe pod <pod-name>	Adjust resources or add more nodes
Node Full	Node has no allocatable resources	Pod status: Pending Events: FailedScheduling	kubectl describe pod <pod-name>	Scale cluster or redistribute workloads

USEFUL COMMANDS SUMMARY

```
kubectl top nodes
kubectl top pods
kubectl describe pod <pod-name>
kubectl describe node <node-name>
kubectl get events --sort-by=.lastTimestamp
kubectl get pod <pod-name> -o wide
```



BEST PRACTICES

- Always set both requests and limits.
- Start with reasonable values and adjust.
- Monitor resource usage continuously.
- Avoid setting limits too close to requests.
- Design applications to handle restarts.



VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



PAGE 11 - PROJECT 10

CREATE PERSISTENT APPLICATION STORAGE



OBJECTIVE

Learn how to provision persistent storage in Kubernetes using PersistentVolume (PV) and PersistentVolumeClaim (PVC), mount it into a Pod, store data, and verify that the data survives Pod recreation.



PROJECT STEPS

- 1 CREATE A PERSISTENTVOLUME (PV)
- 2 CREATE A PERSISTENTVOLUMECLAIM (PVC)
- 3 MOUNT STORAGE INTO A POD
- 4 WRITE PERSISTENT APPLICATION DATA
- 5 DELETE AND RECREATE THE POD
- 6 VERIFY DATA PERSISTS
- 7 TROUBLESHOOT PENDING PVCs

1 CREATE A PERSISTENTVOLUME (PV)

EXAMPLE: LOCAL PATH PV

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  local:
    path: /mnt/data/pv-demo
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - minikube
```

APPLY THE PV

```
kubectl apply -f pv-demo.yaml
```

2 CREATE A PERSISTENTVOLUMECLAIM (PVC)

EXAMPLE: PVC REQUESTING THE PV

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  resources:
    requests:
      storage: 1Gi
```

APPLY THE PVC

```
kubectl apply -f pvc-demo.yaml
```

CHECK PVC STATUS

```
kubectl get pvc pvc-demo
```

PVC STATUS PHASES

- Bound** PVC is bound to a PV and ready to use.
- Pending** PVC is waiting for a suitable PV.
- Lost** PVC cannot find its bound PV.

3 MOUNT STORAGE INTO A POD

EXAMPLE: POD USING THE PVC

```
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: web
      image: nginx
      volumeMounts:
        - name: web-data
          mountPath: /usr/share/nginx/html
  volumes:
    - name: web-data
      persistentVolumeClaim:
        claimName: pvc-demo
```

APPLY THE POD

```
kubectl apply -f pod-with-pvc.yaml
```

CHECK POD STATUS

```
kubectl get pods web-pod
```

4 WRITE PERSISTENT APPLICATION DATA

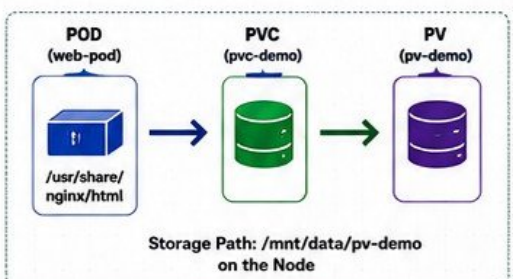
WRITE DATA INSIDE THE POD

```
kubectl exec -it web-pod -- /bin/sh
```

```
echo "Hello Kubernetes Storage" >
/usr/share/nginx/html/index.html
```

```
cat /usr/share/nginx/html/index.html
```

You should see: Hello Kubernetes Storage



5 DELETE AND RECREATE THE POD

DELETE THE POD

```
kubectl delete pod web-pod
```

RECREATE THE POD

```
kubectl apply -f pod-with-pvc.yaml
```

CHECK POD STATUS

```
kubectl get pods web-pod
```

6 VERIFY DATA PERSISTS

ACCESS THE POD AGAIN

```
kubectl exec -it web-pod -- /bin/sh
```

READ THE DATA FILE

```
cat /usr/share/nginx/html/index.html
```

EXPECTED OUTPUT

```
Hello Kubernetes Storage
```

The data survived Pod deletion and recreation!

7 TROUBLESHOOT PENDING PVCs

COMMON CAUSES AND FIXES

CAUSE	HOW TO FIX
No matching PV	Create a PV that matches capacity, accessModes, and storageClassName.
StorageClass mismatch	Ensure PVC storageClassName matches PV storageClassName.
PV already bound	PVC can only bind to an unbound PV.
Insufficient capacity	Increase PV capacity or change PVC request.
AccessModes mismatch	Ensure PV and PVC accessModes are compatible.

CHECK PVC EVENTS

```
kubectl describe pvc pvc-demo
```

CHECK AVAILABLE PVs

```
kubectl get pv
```

USEFUL COMMANDS SUMMARY

```
kubectl get pv          # List all PersistentVolumes
kubectl get pvc         # List all PersistentVolumeClaims
kubectl get pods        # List Pods
kubectl describe pv <name> # Inspect PV details
kubectl describe pvc <name> # Inspect PVC details
kubectl describe pod <name> # Inspect Pod details
kubectl exec -it <pod> -- /bin/sh # Access Pod shell
kubectl logs <pod>      # Check Pod logs
```

★ KEY TAKEAWAYS

- PV is cluster storage, PVC is a request for storage.
- PVC binds to a compatible PV.
- Data remains even if the Pod is deleted.
- AccessModes control how the storage can be used.
- StorageClass helps with dynamic provisioning.
- Always check events when PVCs are Pending.





PAGE 12 - PROJECT 11

RUN JOBS AND SCHEDULED TASKS



OBJECTIVE

Learn how to run one-time tasks and scheduled tasks in Kubernetes using Jobs and CronJobs, monitor their execution, troubleshoot failures, and manage their history and cleanup.



PROJECT STEPS

- 1 CREATE A KUBERNETES JOB
- 2 INSPECT JOB COMPLETION
- 3 VIEW JOB LOGS
- 4 CREATE A CRONJOB
- 5 CONFIGURE A SCHEDULE
- 6 INSPECT JOB HISTORY
- 7 TROUBLESHOOT FAILED JOBS
- 8 CLEAN UP COMPLETED WORKLOADS

1 CREATE A KUBERNETES JOB

EXAMPLE: JOB YAML

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  template:
    spec:
      containers:
        - name: hello
          image: busybox:1.36
          command: ["/bin/sh", "-c", "echo
Hello Kubernetes Job! && date >
/data/output.txt"]
      volumeMounts:
        - name: data
          mountPath: /data
      restartPolicy: Never
      backoffLimit: 3
      ttlSecondsAfterFinished: 60
```

WHAT HAPPENS



KEY POINTS

- ✓ Jobs run Pods to completion.
- ✓ Pods created by a Job do not restart on success.
- ✓ Use restartPolicy: Never for Jobs.
- ✓ ttlSecondsAfterFinished cleans up Job automatically.

2 INSPECT JOB COMPLETION

CHECK JOB STATUS

```
kubectl get jobs
```

EXAMPLE OUTPUT

NAME	COMPLETIONS	DURATION	AGE	STATUS
hello-job	1/1	8s	2m	Complete

CHECK JOB PODS

```
kubectl get pods -l job-name=hello-job
```

3 VIEW JOB LOGS

GET THE POD NAME

```
kubectl get pods -l job-name=hello-job
```

VIEW LOGS

```
kubectl logs <pod-name>
```

FOLLOW LOGS

```
kubectl logs -f <pod-name>
```

EXAMPLE LOG OUTPUT

```
Hello Kubernetes Job!
Wed May 15 10:30:00 UTC 2024
```

4 CREATE A CRONJOB

EXAMPLE: CRONJOB YAML

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: hello-cronjob
spec:
  schedule: "*/5 * * * *" # Every 5 minutes
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 2
  jobTemplate:
    spec:
      template:
        spec:
          containers:
            - name: hello
              image: busybox:1.36
              command: ["/bin/sh", "-c", "date >
/data/cron.txt"]
          restartPolicy: Never
```

APPLY THE CRONJOB

```
kubectl apply -f cronjob.yaml
```

VERIFY CRONJOB

```
kubectl get cronjobs
```

5 CONFIGURE A SCHEDULE

CRON SCHEDULE FORMAT

```
* * * * *
MINUTE HOUR DAY OF MONTH DAY OF WEEK
```

EXAMPLES

/ 5 * * *	Every 5 minutes
0 * * * *	Every hour at minute 0
30 2 * * *	Every day at 2:30 AM
0 0 * * 0	Every Sunday at midnight

6 INSPECT JOB HISTORY

LIST JOBS CREATED BY THE CRONJOB

```
kubectl get jobs
```

FILTER BY CRONJOB NAME

```
kubectl get jobs -l cronjob-name=hello-cronjob
```

EXAMPLE OUTPUT

NAME	COMPLETIONS	DURATION	AGE	STATUS
hello-cronjob-28277100	1/1	6s	10m	Complete
hello-cronjob-28277040	1/1	5s	15m	Complete
hello-cronjob-28277020	1/1	7s	20m	Failed

7 TROUBLESHOOT FAILED JOBS

CHECK FAILED JOBS

```
kubectl get jobs --field-selector=status.failed=0
```

DESCRIBE THE JOB

```
kubectl describe job <job-name>
```

VIEW PODS CREATED BY THE JOB

```
kubectl get pods -l job-name=<job-name>
```

VIEW POD LOGS (USE --PREVIOUS IF NEEDED)

```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
```

8 CLEAN UP COMPLETED WORKLOADS

DELETE A JOB

```
kubectl delete job hello-job
```

DELETE A CRONJOB

```
kubectl delete cronjob hello-cronjob
```

DELETE OLD JOBS (KEEP HISTORY LIMITS SMALL)

```
Settings: successfulJobsHistoryLimit,
failedJobsHistoryLimit
```



USEFUL COMMANDS SUMMARY

```
kubectl get jobs
kubectl describe job <name>
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
kubectl get pods -l job-name=<job-name>
kubectl get cronjobs
kubectl describe cronjob <name>
kubectl delete job <name>
kubectl delete cronjob <name>
```



BEST PRACTICES

- Keep job payloads idempotent.
- Set reasonable backoffLimit for retries.
- Limit job history to avoid resource waste.
- Store logs or results in persistent storage if needed.
- Monitor CronJobs regularly.
- Clean up old Jobs and Pods to keep clusters healthy.





PAGE 13 - PROJECT 12

CONTROL WORKLOAD PLACEMENT

VERIQTA



VERIQTA

VERIQTA

VERIQTA



OBJECTIVE

Learn how to control where Pods run in a Kubernetes cluster using labels, node selectors, node affinity, taints, and tolerations.



PROJECT STEPS

- 1 ADD LABELS TO NODES
- 2 USE NODESELECTOR
- 3 INTRODUCE NODE AFFINITY
- 4 UNDERSTAND TAINTS AND TOLERATIONS
- 5 SCHEDULE A WORKLOAD ONTO A SPECIFIC NODE
- 6 DIAGNOSE A POD STUCK IN PENDING
- 7 INSPECT SCHEDULER EVENTS

1 ADD LABELS TO NODES

VIEW NODES

```
kubectl get nodes --show-labels
```

ADD LABELS

```
kubectl label node node1 disktype=ssd
```

```
kubectl label node node2 disktype=hdd
```

```
kubectl label node node3 env=prod
```

EXAMPLE NODES WITH LABELS

NODE NAME	DISKTYPE	ENV	KUBERNETES.IO/HOSTNAME
node1	ssd	prod	node1
node2	hdd	staging	node2
node3	ssd	prod	node3

✓ Labels help the scheduler make placement decisions.

2 USE NODESELECTOR

EXAMPLE: NODESELECTOR YAML

```
spec:
  template:
    spec:
      nodeSelector:
        disktype: ssd
```

APPLY THE WORKLOAD

```
kubectl apply -f nginx-nodeselector.yaml
```

VERIFY PLACEMENT

```
kubectl get pods -o wide
```

3 INTRODUCE NODE AFFINITY

EXAMPLE: NODE AFFINITY (PREFERRED)

```
spec:
  template:
    spec:
      affinity:
        nodeAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              preference:
                matchExpressions:
                  - key: env
                    operator: In
                    values:
                      - prod
```

APPLY THE WORKLOAD

```
kubectl apply -f nginx-affinity.yaml
```

4 TAINTS AND TOLERATIONS

ADD TAINT TO A NODE

```
kubectl taint node node2 dedicated=backend:NoSchedule
```

EXAMPLE: TOLERATION IN POD YAML

```
spec:
  template:
    spec:
      tolerations:
        - key: "dedicated"
          operator: "Equal"
          value: "backend"
          effect: "NoSchedule"
```

EFFECT

Only Pods with the matching toleration can be scheduled on the tainted node.

5 SCHEDULE A WORKLOAD ONTO A SPECIFIC NODE

EXAMPLE: FORCE POD ON A SPECIFIC NODE

```
spec:
  template:
    spec:
      nodeName: node1 # Schedules directly to node1
```

APPLY THE WORKLOAD

```
kubectl apply -f pod-on-node.yaml
```

SCHEDULING STRATEGY OVERVIEW



- ✓ Node Selectors (Hard rule)
- ✓ Node Affinity (Soft rule)
- ✓ Taints & Tolerations
- ✓ Resources Available
- ✓ Other Constraints

6 DIAGNOSE A POD STUCK IN PENDING

CHECK POD STATUS

```
kubectl get pods -o wide
```

DESCRIBE THE POD

```
kubectl describe pod <pod-name>
```

COMMON REASONS FOR PENDING

- No matching node selector or affinity
- Taints not tolerated
- Insufficient CPU or memory
- Node not ready
- PodDisruptionBudget
- PersistentVolumeClaim pending

LOOK FOR EVENTS

```
kubectl describe pod <pod-name>
```

Check the Events section for scheduling failure messages.



7 INSPECT SCHEDULER EVENTS

VIEW RECENT EVENTS

```
kubectl get events --sort-by=.lastTimestamp
```

FILTER SCHEDULER EVENTS

```
kubectl get events | grep -i scheduler
```

EXAMPLE EVENT MESSAGE

0/3 nodes are available: 1 node(s) didn't match node selector, 1 node(s) had taint (dedicated=backend:NoSchedule), 1 node(s) had insufficient memory.

This tells you exactly why the Pod cannot be scheduled.

★ QUICK REFERENCE COMMANDS

- `kubectl get nodes --show-labels` # View node labels
- `kubectl label node <node> <key>=<value>` # Add label
- `kubectl describe node <node>` # View node details
- `kubectl taint node <node> <key>=<value>:effect` # Add taint
- `kubectl get pods -o wide` # Check Pod placement
- `kubectl describe pod <pod>` # Troubleshoot Pending
- `kubectl get events --sort-by=.lastTimestamp` # View scheduler events

💡 KEY TAKEAWAYS

- Labels help organize and select nodes.
- NodeSelector is a strict requirement.
- Node Affinity is a preference.
- Taints repel Pods.
- Tolerations allow Pods with matching keys.
- Always inspect events to understand scheduling failures.

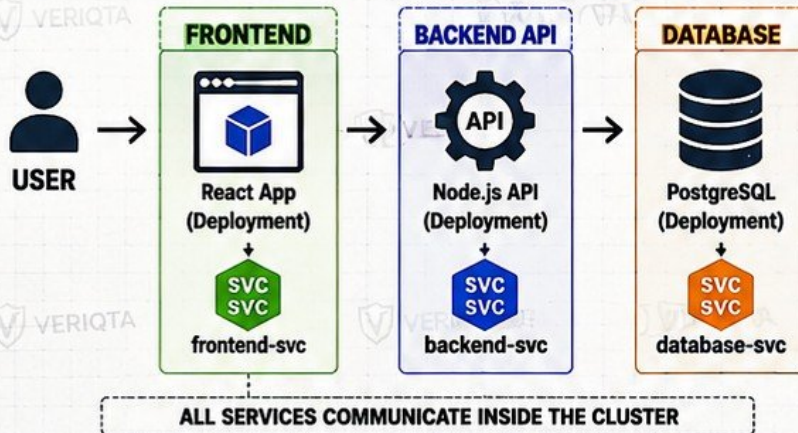
⚙️ BEST PRACTICES

- Use labels consistently across nodes.
- Prefer node affinity over nodeName.
- Use taints for node isolation.
- Avoid over-constraining workloads.
- Monitor scheduler events regularly.



PROJECT 13: BUILD A MULTI-TIER APPLICATION

BUILD A SMALL REALISTIC ENVIRONMENT



TASKS

- ✓ DEPLOY MULTIPLE WORKLOADS
- ✓ CREATE INTERNAL SERVICES
- ✓ CONFIGURE ENVIRONMENT VARIABLES
- ✓ CONNECT FRONTEND AND BACKEND
- ✓ CONNECT BACKEND AND DATABASE
- ✓ TEST SERVICE DISCOVERY
- ✓ DIAGNOSE BROKEN SERVICE COMMUNICATION

1. WORKLOADS AND SERVICES OVERVIEW

TIER	WORKLOAD	CONTAINER IMAGE	SERVICE	PORT	DESCRIPTION
FRONTEND	frontend-deployment	nginx:latest (React Build)	frontend-svc	80	Serves the web UI
BACKEND API	backend-deployment	node:18-alpine	backend-svc	3000	REST API for the application
DATABASE	database-deployment	postgres:15-alpine	database-svc	5432	PostgreSQL database

SERVICE DISCOVERY

FRONTEND POD (Calls)

BACKEND SERVICE
backend-svc:3000

DATABASE SERVICE
database-svc:5432

KUBERNETES DNS RESOLVES
SERVICES AUTOMATICALLY

2. ENVIRONMENT VARIABLES EXAMPLE

FRONTEND .ENV (CONFIGMAP OR ENV)

REACT_APP_API_URL=http://backend-svc:3000
PORT=80

BACKEND .ENV (FROM SECRET OR CONFIGMAP)

PORT=3000
DB_HOST=database-svc
DB_PORT=5432
DB_USER=appuser
DB_PASSWORD=StrongPassword123
DB_NAME=appdb

3. TEST COMMUNICATION

FROM FRONTEND POD

kubectl exec -it <frontend-pod> -- sh
wget -qO- http://backend-svc:3000/health

FROM BACKEND POD

kubectl exec -it <backend-pod> -- sh
wget -qO- http://database-svc:5432

DNS LOOKUP TEST

kubectl exec -it <backend-pod> -- sh
nslookup backend-svc
nslookup database-svc

4. DIAGNOSE BROKEN SERVICE COMMUNICATION

- ⚠ SERVICE HAS NO ENDPOINTS
kubectl get endpoints
- ⚠ WRONG SERVICE SELECTOR
kubectl describe svc <service-name>
- ⚠ POD IS NOT READY
kubectl get pods -o wide
- ⚠ NETWORK POLICY BLOCKING
kubectl get networkpolicy -A
- ⚠ ENVIRONMENT VARIABLE INCORRECT
kubectl exec -it <pod> -- printenv
- ⚠ APPLICATION NOT RUNNING
kubectl logs <pod-name>

5. SAMPLE MANIFEST STRUCTURE

/manifests

frontend
deployment.yaml
service.yaml

backend
deployment.yaml
service.yaml
configmap.yaml

database
deployment.yaml
service.yaml
pvc.yaml
secret.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: node:18-alpine
          ports:
            - containerPort: 3000
          env:
            - name: DB_HOST
              value: "database-svc"
            - name: PORT
              value: "3000"
```

WHAT YOU WILL LEARN

- ✓ MULTI-TIER ARCHITECTURE DEPLOYMENT
- ✓ INTERNAL SERVICE COMMUNICATION
- ✓ SERVICE DISCOVERY
- ✓ CONFIGURATION MANAGEMENT
- ✓ DEPENDENCY MANAGEMENT BETWEEN SERVICES
- ✓ REAL-WORLD TROUBLESHOOTING

TIP
ALWAYS VERIFY EACH LAYER
BEFORE MOVING TO THE NEXT.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 14: TROUBLESHOOT A CRASHLOOPBACKOFF

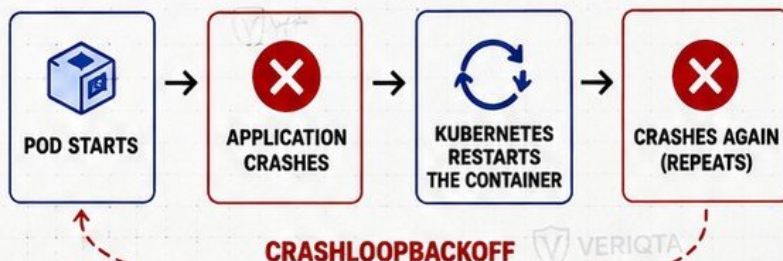
START WITH AN INTENTIONALLY BROKEN APPLICATION

AN APPLICATION IS DEPLOYED BUT THE POD IS CRASHING REPEATEDLY.

SYMPTOM



- POD STATUS: RUNNING (RESTARTING)
- RESTART COUNT INCREASING
- APPLICATION NOT AVAILABLE



INVESTIGATE THE FOLLOWING

- 1 POD STATUS
- 2 CONTAINER STATE
- 3 RESTART COUNT
- 4 APPLICATION LOGS
- 5 PREVIOUS CONTAINER LOGS
- 6 POD EVENTS
- 7 CONFIGURATION
- 8 CONTAINER COMMAND

USE THESE COMMANDS



kubectl describe pod <pod-name>
SHOWS POD DETAILS, CONTAINER STATE, RESTART COUNT, EVENTS, AND WARNINGS.



kubectl logs <pod-name>
SHOWS THE CURRENT CONTAINER LOGS. USEFUL FOR APPLICATION ERRORS.



kubectl logs <pod-name> --previous
SHOWS THE LOGS FROM THE PREVIOUS CONTAINER INSTANCE (BEFORE THE CRASH).

EXAMPLE: kubectl describe pod my-app-pod

```

Name:          my-app-pod
Namespace:     default
Status:        Running (Restarting)
Restart Count: 7
Containers:
  app:
    State:      Waiting
    Reason:     CrashLoopBackOff
    Last State: Terminated
    Reason:     Error
    Exit Code:  1
Events:
  Warning   BackOff    12s (x7 over 2m)  kubelet
  Back-off restarting failed container
  
```

COMMON ROOT CAUSES

- INVALID CONFIGURATION
- MISSING CONFIG FILE OR ENVIRONMENT
- WRONG COMMAND OR ARGUMENTS
- APPLICATION ERROR OR BUG
- DEPENDENCY NOT AVAILABLE
- INSUFFICIENT RESOURCES
- INVALID SECRET OR CONFIGMAP REFERENCE

TROUBLESHOOTING STEPS

- 1 CHECK POD STATUS AND RESTART COUNT
- 2 VIEW CURRENT LOGS
- 3 VIEW PREVIOUS LOGS
- 4 CHECK POD EVENTS
- 5 REVIEW CONFIGURATION, ENV, AND SECRETS
- 6 VERIFY CONTAINER COMMAND AND ARGS
- 7 IDENTIFY THE ROOT CAUSE
- 8 FIX THE ISSUE AND REDEPLOY



GOAL

FIND THE ROOT CAUSE AND RESTORE THE APPLICATION TO A STABLE RUNNING STATE.



**ALWAYS READ THE LOGS, EVENTS, AND CONFIGURATION CAREFULLY.
DON'T GUESS. INVESTIGATE!**

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 15: TROUBLESHOOT IMAGEPULLBACKOFF AND PENDING PODS

CREATE SEVERAL INTENTIONAL FAILURES. IDENTIFY THE CAUSE AND FIX EACH ONE.

PODS MAY FAIL TO START FOR MANY REASONS. YOUR JOB IS TO FIND OUT WHY.



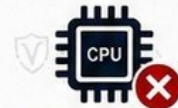
1. INVALID
CONTAINER IMAGE



2. INCORRECT
IMAGE TAG



3. MISSING IMAGE
CREDENTIALS



4. EXCESSIVE
RESOURCE REQUEST



5. SCHEDULING
PROBLEM

COMMON POD STATUS YOU WILL SEE

STATUS	WHAT IT MEANS
● ImagePullBackOff	Kubernetes cannot pull the container image.
● ErrImagePull	There was an error pulling the image.
● Pending	Pod is waiting for resources or scheduling.
● ContainerCreating	Kubernetes is preparing the container.

USE THESE COMMANDS TO INVESTIGATE

- >_** `kubectl describe pod <pod-name>`
Shows events, errors, scheduling details and reasons for failure.
- >_** `kubectl get pod <pod-name> -o wide`
Shows node selection and scheduling info.
- >_** `kubectl logs <pod-name>`
Shows application logs if the container started.
- >_** `kubectl get events --sort-by=.metadata.creationTimestamp`
Shows recent cluster events.

5 INTENTIONAL FAILURES TO PRACTICE

#	SCENARIO	EXAMPLE MANIFEST ISSUE	EXPECTED STATUS	WHAT TO CHECK	HOW TO FIX
1	INVALID CONTAINER IMAGE	<code>image: myrepo/does-not-exist:latest</code>	ImagePullBackOff ErrImagePull	<code>kubectl describe pod</code> - Image name spelling - Repository exists?	- Correct the image name - Or use a valid image
2	INCORRECT IMAGE TAG	<code>image: nginx:wrongtag</code>	ImagePullBackOff	<code>kubectl describe pod</code> - Check tag spelling - Does the tag exist?	- Use a valid tag (e.g., <code>nginx:1.25</code>) - Or use <code>nginx:latest</code>
3	MISSING IMAGE CREDENTIALS	<code>image: private-registry/app:1.0</code> # No <code>imagePullSecrets</code>	ImagePullBackOff	<code>kubectl describe pod</code> - Look for unauthorized or denied errors	- Create <code>docker-registry secret</code> - Add <code>imagePullSecrets</code> to the Pod spec
4	EXCESSIVE RESOURCE REQUEST	<code>resources:</code> <code>requests:</code> <code>memory: "16Gi"</code> <code>cpu: "8"</code>	Pending (Insufficient resources)	<code>kubectl describe pod</code> - Check events - Insufficient cpu/memory on all nodes	- Reduce requests/limits - Or add more resources to the cluster
5	SCHEDULING PROBLEM	<code>nodeSelector:</code> <code>disktype: ssd</code> # No node has this label	Pending (No nodes available)	<code>kubectl describe pod</code> - Look for 0/.. nodes are available - Check node labels and taints	- Use correct labels - Add tolerations if needed - Or remove invalid <code>nodeSelector</code>



YOUR GOAL

DETERMINE WHY EACH POD CANNOT START AND CORRECT THE CONFIGURATION UNTIL THE POD RUNS SUCCESSFULLY.



REMEMBER

- READ THE EVENTS CAREFULLY.
- THE ROOT CAUSE IS USUALLY IN THE POD EVENTS.
- DO NOT GUESS. ALWAYS INVESTIGATE.

VERIFICATION CHECKLIST

- ☒ POD IS RUNNING
- ☒ READY STATUS IS TRUE
- ☒ POD IS ON THE EXPECTED NODE
- ☒ NO WARNINGS OR ERRORS
- ☒ APPLICATION IS ACCESSIBLE



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

VERIQTA SOCIAL

PROJECT 16: TROUBLESHOOT KUBERNETES NETWORKING

BREAK APPLICATION CONNECTIVITY INTENTIONALLY. FIND THE ROOT CAUSE AND FIX IT.

THE GOAL



RESTORE APPLICATION CONNECTIVITY WITHOUT REDEPLOYING THE ENTIRE ENVIRONMENT.



DATABASE SERVICE

INVESTIGATE THESE AREAS

POD IP ADDRESSES	SERVICE CONFIGURATION	SELECTORS	ENDPOINTS	TARGET PORTS	DNS RESOLUTION	CONTAINER CONNECTIVITY
 10.244.x.x						

PRACTICE THESE COMMANDS

<pre>> kubectl get svc</pre> LIST ALL SERVICES AND THEIR CLUSTER IPs AND PORTS.	<pre>> kubectl get endpoints</pre> VIEW ENDPOINTS (POD IPs AND PORTS) FOR A SERVICE.	<pre>> kubectl exec -it <pod> -- sh</pre> EXECUTE A SHELL INSIDE A POD TO TEST CONNECTIVITY.	<pre>> nslookup <service-name> <namespace>.svc .cluster.local</pre> TEST DNS RESOLUTION INSIDE THE CLUSTER.	<pre>> curl http://<service-name>:<port>/<path></pre> TEST HTTP CONNECTIVITY BETWEEN SERVICES.
---	--	--	---	--

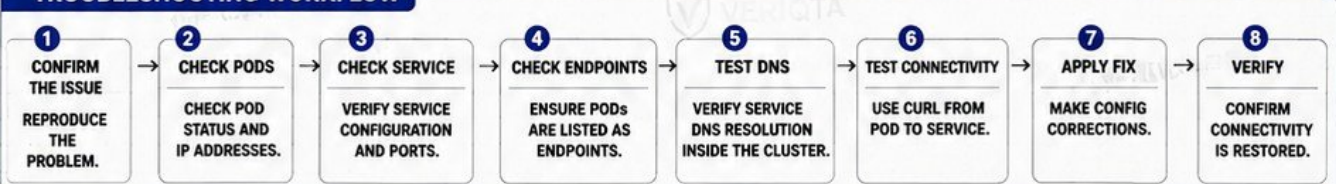
COMMON NETWORK ISSUES YOU WILL ENCOUNTER

- SERVICE SELECTOR DOES NOT MATCH POD LABELS
- ENDPOINTS LIST IS EMPTY
- WRONG TARGET PORT OR SERVICE PORT
- DNS NAME RESOLUTION FAILURE
- POD CANNOT REACH SERVICE OR ANOTHER POD
- NETWORK POLICY BLOCKING TRAFFIC
- APPLICATION LISTENING ON WRONG PORT
- WRONG NAMESPACE REFERENCES

HOW TO CHECK AND FIX

- CHECK SERVICE SELECTOR AND POD LABELS
→ RECTIFY SELECTOR OR LABELS
- CHECK ENDPOINTS
→ FIX SELECTOR OR POD READINESS
- CHECK SERVICE AND CONTAINER PORTS
→ MATCH PORTS CORRECTLY
- TEST DNS WITH NSLOOKUP
→ VERIFY SERVICE NAME AND NAMESPACE
- TEST CONNECTIVITY WITH CURL OR TELNET
→ VERIFY NETWORK AND SERVICE ACCESS
- CHECK NETWORK POLICIES
→ UPDATE POLICY RULES
- VERIFY CONTAINER LISTENING PORT
→ CORRECT APPLICATION CONFIG
- VERIFY NAMESPACE IN MANIFESTS
→ USE CORRECT NAMESPACE

TROUBLESHOOTING WORKFLOW



TIP
ALWAYS TEST FROM THE SAME NETWORK PATH AS YOUR APPLICATION. TESTING FROM YOUR LOCAL MACHINE MAY NOT REFLECT THE REAL ISSUE.



REMEMBER
THE GOAL IS TO RESTORE CONNECTIVITY USING OBSERVABILITY, NOT GUESSWORK.



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

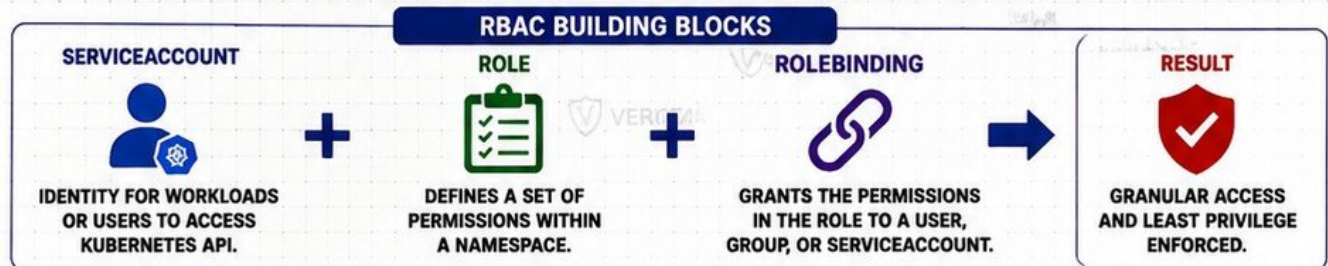


/VERIQTA

VERIQTA SOCIAL

PROJECT 17: IMPLEMENT BASIC RBAC

CONTROL WHO CAN DO WHAT IN YOUR CLUSTER.



STEP 1: CREATE THE RBAC COMPONENTS (EXAMPLE)

1. CREATE SERVICEACCOUNT

```

apiVersion: v1
kind: ServiceAccount
metadata:
  name: app-reader
  namespace: demo
  
```



2. CREATE ROLE

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: demo
rules:
- apiGroups: [""]
  resources: ["pods", "services"]
  verbs: ["get", "list", "watch"]
  
```



3. CREATE ROLEBINDING

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: app-reader-binding
  namespace: demo
subjects:
- kind: ServiceAccount
  name: app-reader
  namespace: demo
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
  
```



WHO CAN WE BIND?

- USER
- GROUP
- SERVICEACCOUNT

EXAMPLE: WHAT THIS RBAC ALLOWS

The ServiceAccount "app-reader" in the "demo" namespace can now:

- ✓ VIEW LIST OF PODS
- ✓ VIEW LIST OF SERVICES
- ✗ CANNOT CREATE, UPDATE OR DELETE ANYTHING
- ✗ CANNOT ACCESS OTHER NAMESPACES

NAMESPACE-SCOPED ACCESS

This Role and RoleBinding exist only in the "demo" namespace.

The ServiceAccount CANNOT access resources in other namespaces.



STEP 2: TEST PERMISSIONS WITH kubectl auth can-i

TEST	COMMAND	EXPECTED RESULT	WHAT IT MEANS
✓ ALLOWED: LIST PODS	<code>kubectl auth can-i list pods \</code> <code>--as system:serviceaccount:demo:app-reader</code> <code>-n demo</code>	yes	The ServiceAccount can list Pods in the "demo" namespace.
✓ ALLOWED: GET SERVICES	<code>kubectl auth can-i get services \</code> <code>--as system:serviceaccount:demo:app-reader</code> <code>-n demo</code>	yes	The ServiceAccount can get Services in the "demo" namespace.
✗ DENIED: CREATE PODS	<code>kubectl auth can-i create pods \</code> <code>--as system:serviceaccount:demo:app-reader</code> <code>-n demo</code>	no	The ServiceAccount cannot create Pods.
✗ DENIED: DELETE SERVICES	<code>kubectl auth can-i delete services \</code> <code>--as system:serviceaccount:demo:app-reader</code> <code>-n demo</code>	no	The ServiceAccount cannot delete Services.
✗ DENIED: ACCESS OTHER NAMESPACE	<code>kubectl auth can-i list pods \</code> <code>--as system:serviceaccount:demo:app-reader</code> <code>-n kube-system</code>	no	The ServiceAccount cannot access other namespaces.



BEST PRACTICES

- ✓ GRANT ONLY THE MINIMUM PERMISSIONS REQUIRED.
- ✓ USE NAMESPACE-SCOPED ROLES WHEN POSSIBLE.
- ✓ AVOID USING CLUSTER-ADMIN FOR WORKLOADS.
- ✓ REVIEW ROLES AND ROLEBINDINGS REGULARLY.
- ✓ FOLLOW THE PRINCIPLE OF LEAST PRIVILEGE.



WHY NOT GIVE ADMIN PERMISSIONS?

- HIGH RISK: A COMPROMISED WORKLOAD CAN TAKE OVER THE CLUSTER.
- HARDER TO AUDIT AND CONTROL.
- VIOLATES SECURITY BEST PRACTICES.
- INCREASES CHANCE OF ACCIDENTAL OR MALICIOUS DAMAGE.



REMEMBER: RBAC IS YOUR FIRST LINE OF DEFENSE. GIVE JUST ENOUGH ACCESS FOR THE JOB, NOTHING MORE.

VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA

PROJECT 18: PERFORM A JUNIOR ADMINISTRATOR CLUSTER HEALTH CHECK

TREAT THE CLUSTER AS IF YOU HAVE JUST RECEIVED AN OPERATIONAL SUPPORT TICKET.

WHAT YOU MUST CHECK

1  NODES	2  NAMESPACES	3  PODS	4  DEPLOYMENTS	5  SERVICES	6  EVENTS
7  RESOURCE USAGE	8  FAILED WORKLOADS	9  RESTARTING CONTAINERS	10  STORAGE	11  CONFIGURATION	12  RECENT APPLICATION FAILURES

USE THESE COMMANDS TO CHECK THE CLUSTER

CHECK ITEM	COMMAND	CHECK ITEM	COMMAND
1 NODES	<code>kubectl get nodes -o wide</code>	7 RESOURCE USAGE	<code>kubectl top nodes</code> <code>kubectl top pods -A</code>
2 NAMESPACES	<code>kubectl get namespaces</code>	8 FAILED WORKLOADS	<code>kubectl get pods -A -f field-selector=status.phase!=Running</code>
3 PODS	<code>kubectl get pods -A -o wide</code>	9 RESTARTING CONTAINERS	<code>kubectl get pods -A -o custom-columns=</code> <code>NAMESPACE:.metadata.namespace,NAME:.metadata.name,RESTARTS:</code> <code>.status.containerStatuses[0].restartCount</code>
4 DEPLOYMENTS	<code>kubectl get deploy -A</code>	10 STORAGE	<code>kubectl get pvc -A</code> <code>kubectl get pv</code>
5 SERVICES	<code>kubectl get svc -A</code>	11 CONFIGURATION	<code>kubectl get cm -A</code> <code>kubectl get secret -A</code>
6 EVENTS	<code>kubectl get events -A</code> <code>--sort-by=.metadata.creationTimestamp</code>	12 RECENT FAILURES	<code>kubectl logs <pod> -n <ns> --tail=100</code> <code>kubectl describe pod <pod> -n <ns></code>

COMMON HEALTH INDICATORS

-  **HEALTHY**
All resources are running as expected.
-  **DEGRADED**
Some resources have issues but the cluster is functional.
-  **CRITICAL**
Multiple workloads are failing or the cluster is unstable.
-  **UNHEALTHY**
Services are down or the cluster cannot run workloads.

THINGS TO LOOK FOR

- Nodes NotReady or Unreachable
- Pods in Pending, CrashLoopBackOff, ImagePullBackOff, or Error state
- High CPU or Memory usage
- Frequent container restarts
- Failed Deployments or Jobs
- Storage issues or Pending PVCs
- DNS or Service connectivity failures
- Recent errors in Events
- Misconfigurations in ConfigMaps, Secrets, or Environment Variables

QUICK CLUSTER SCORECARD (EXAMPLE)

COMPONENT	STATUS	NOTES
NODES		All nodes Ready
PODS		3 pods not running
DEPLOYMENTS		All up-to-date
SERVICES		All services OK
STORAGE		1 PVC Pending
EVENTS		ImagePull errors
RESOURCE USAGE		High memory on node-2
OVERALL HEALTH	 DEGRADED	

CREATE A SHORT CLUSTER HEALTH REPORT

1  HEALTHY COMPONENTS • List components working correctly • Notes on stability and performance	2  PROBLEMS DISCOVERED • List all issues found • Include error messages and symptoms	3  ROOT CAUSES • Why did the issue occur? • Misconfiguration? • Resource limits? • Network? • Missing dependencies?	4  ACTIONS TAKEN • What did you change? • How did you fix it? • Commands or YAML updates made	5  REMAINING RISKS • Issues not yet fixed • Potential risks • Recommendations
---	---	--	--	--

BEST PRACTICES

-  CHECK THE CLUSTER DAILY
-  ALWAYS READ EVENTS AND LOGS
-  INVESTIGATE BEFORE RESTARTING ANYTHING
-  FIX THE ROOT CAUSE, NOT THE SYMPTOM
-  DOCUMENT WHAT YOU FIND AND FIX
-  AUTOMATE REPEATED CHECKS WHERE POSSIBLE



TIP

A GOOD ADMINISTRATOR FINDS PROBLEMS EARLY BEFORE USERS REPORT THEM.



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



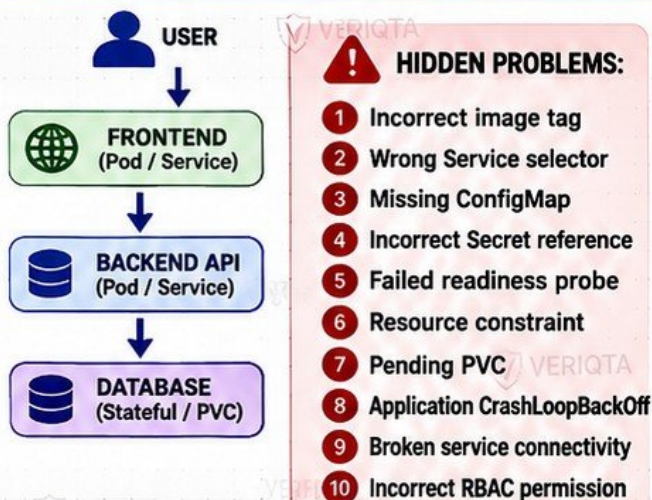
/VERIQTA

VERIQTA SOCIAL

FINAL PROJECT: RECOVER A BROKEN KUBERNETES ENVIRONMENT

RESTORE A FAILING MULTI-TIER APPLICATION TO FULL HEALTH.

THE BROKEN ENVIRONMENT



LEARNER TASKS

- INSPECT THE CLUSTER WITHOUT BEING TOLD WHERE THE FAILURES ARE.
- IDENTIFY UNHEALTHY WORKLOADS.
- READ EVENTS AND LOGS.
- TRACE APPLICATION DEPENDENCIES.
- DIAGNOSE EACH FAILURE.
- CORRECT THE YAML MANIFESTS.
- RESTORE APPLICATION CONNECTIVITY.
- VERIFY APPLICATION HEALTH.
- DOCUMENT THE ROOT CAUSES.
- PRODUCE A FINAL ADMINISTRATOR INCIDENT REPORT.

TROUBLESHOOTING WORKFLOW



1. OBSERVE

- `kubectl get all`
- `kubectl describe`
- `kubectl get events`



2. ANALYZE

- Check logs
- Identify root cause
- Trace dependencies



3. FIX

- Correct YAML
- Apply changes
- Redeploy



4. VERIFY

- Test connectivity
- Confirm recovery
- Document results

COMMON FAILURE SIGNS

- CrashLoopBackOff Pods
- ImagePullBackOff Pods
- Pending PVCs
- Service not routing traffic
- Permission denied errors

ADMIN CHECK COMMANDS

- ```
>_ kubectl get all -A
>_ kubectl describe pod <pod>
>_ kubectl logs <pod>
>_ kubectl get events -A
>_ kubectl auth can-i
```

## FINAL OUTCOME



- ALL WORKLOADS RUNNING
- SERVICE CONNECTIVITY RESTORED
- APPLICATION HEALTHY
- INCIDENT REPORT COMPLETE



**TIP:** A JUNIOR KUBERNETES ADMINISTRATOR DOES NOT JUST DEPLOY — HE OBSERVES, TROUBLESHOOTS AND RESTORES SERVICE HEALTH.

## VERIQTA SOCIAL



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA



/VERIQTA